

Multi-GPU implementation and optimization of High-order shock capturing schemes in the in- house code VYOM for Heterogeneous HPC Architectures

NSM Month-Long Hackathon in Computational Fluid Dynamics (CFD)

Organized jointly by IIT Bombay and C-DAC (Under NSM)

AARAMBH

Name of the Team Members

Gourab Koley
Gagan Wadhwa
Vishal Umapati Choudhari

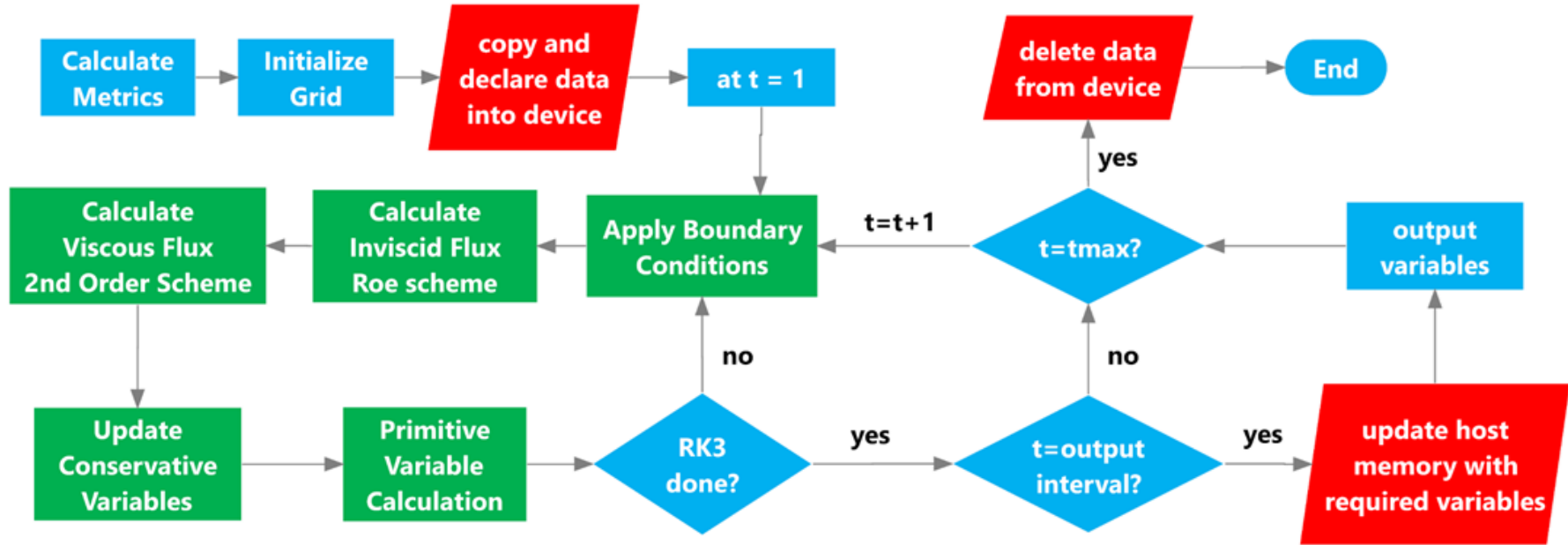
Name of Hackathon Mentor

Mandar Gurav

Name of Hackathon Leader

Dr. Rajesh Ranjan

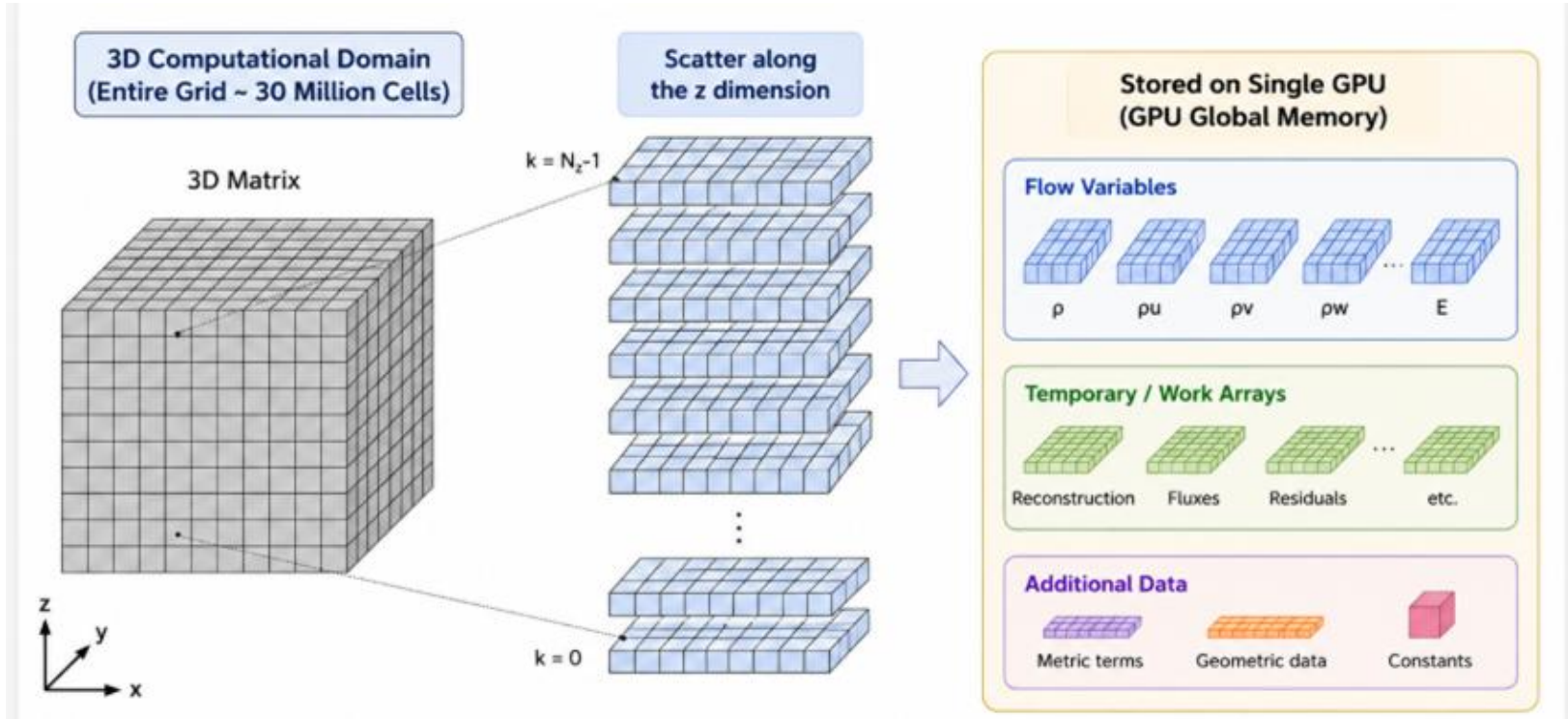
VYOM - 3D



Roe solver GPU flowchart. **Blue** indicates CPU operations, **green** indicates GPU operations and **red** indicates H2D and D2H transfers

Issues: Several H2D and D2H transfer, Multinode communication from D2D, Optimization

Slab/ 1D Domain Decomposition



Problem Statements

- Large-scale simulations are computationally expensive and underutilize GPU resources.
- Implement MPI-based slab decomposition and OpenACC/CUDA acceleration.
- Evaluate performance through scaling studies and optimize using NVIDIA Nsight profiling tools.
- Implementation and Optimization of the VYOM Code for Multi-GPU Architectures

Objective/ Relevance and Impact

- Implementation and evaluation of Slab decomposition strategies for improved scalability.
Ex: : Lid driven Cavity
- Multi-GPU implementation and acceleration of VYOM on HPC systems
- To develop efficient parallelization and memory optimization strategies for the code development.
- Detailed performance evaluation, speedup analysis, scalability assessment, and parallel-efficiency benchmarking across heterogeneous HPC systems.

Relevance and Impact

- The proposed framework is aimed at addressing large-scale aerospace, turbomachinery, and compressible flow applications where both high numerical accuracy and computational efficiency are critical.

Proposed Methodology

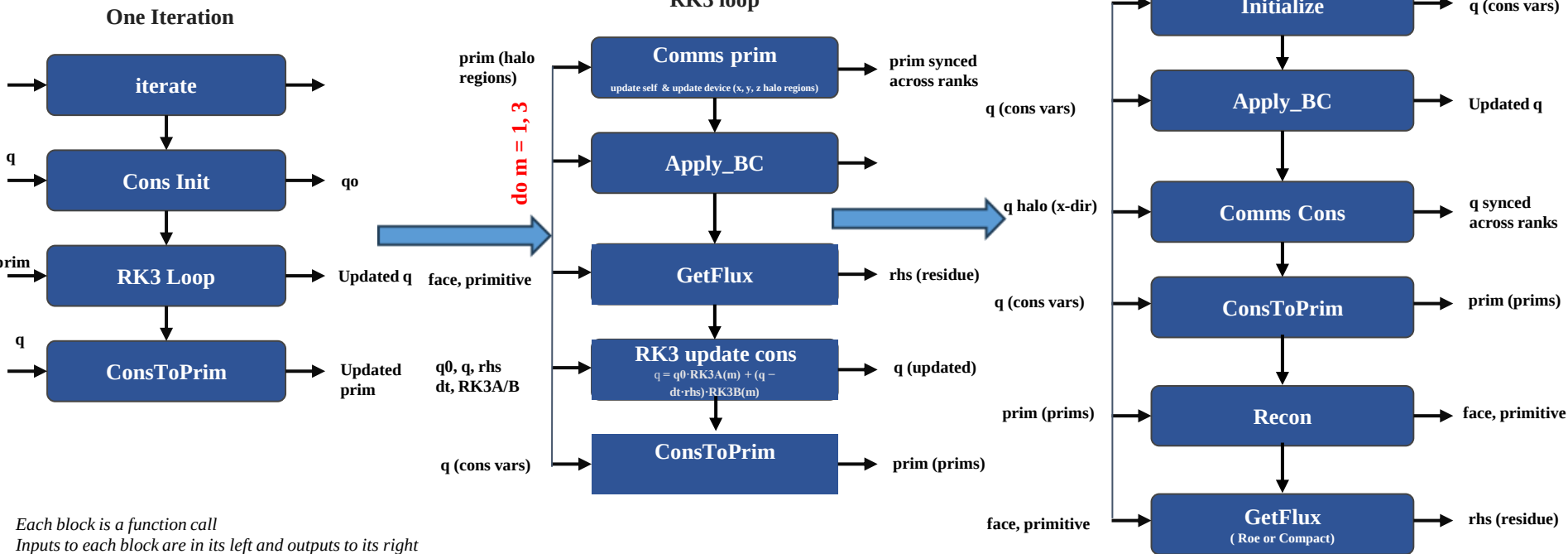
- Analyze the existing CFD solver and establish baseline performance metrics
- Implement slab-based domain decomposition for structured 3D computational grids.
- Develop MPI-based halo exchange for distributed-memory parallelization.
- Port computationally intensive kernels to GPUs using OpenACC/CUDA.
- Extend the implementation to support multi-GPU execution.
- Optimize communication and computation overlap to improve scalability.
- Profile and optimize using the NVIDIA Nsight tools.
- Perform validation, benchmarking, and strong/weak scaling studies.

Computational Requirements

Parameter	Estimate
Grid Size	25 - 30 M Cells
Simulations	3D
Memory Requirement	40 - 50 GB per simulation (depending on scheme)
GPU Required	2 GPU Nodes (H100/A100)
Time Steps	10000 - 100000

Baseline Profiling

Original code - Baseline



Lid Driven Cavity: Multi – GPU Slab Decomposition, 301^3 Mesh, 100 iterations

Time per 100 iterations:: 115.2 s

Speedup = 1.0 x

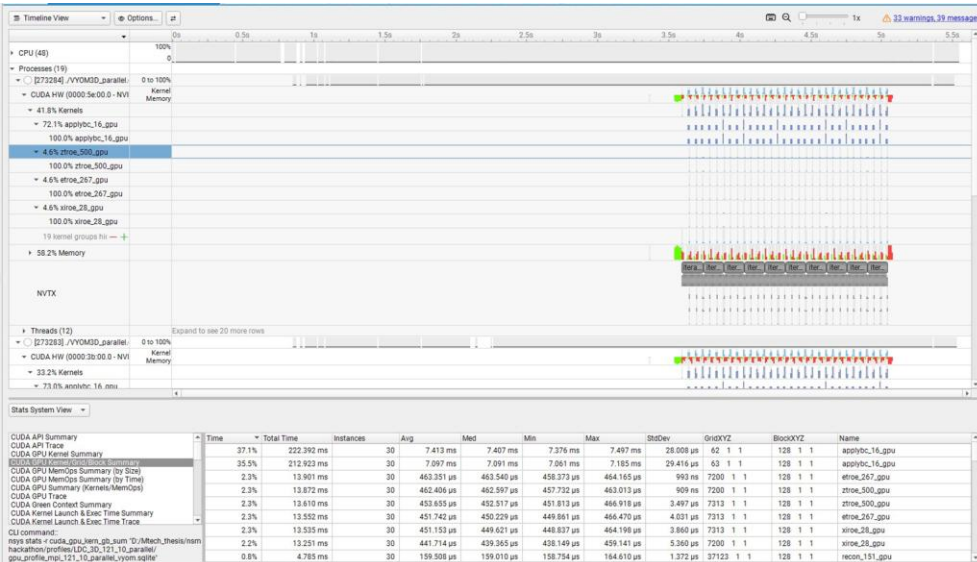
Original Code - Baseline Parallel Architecture



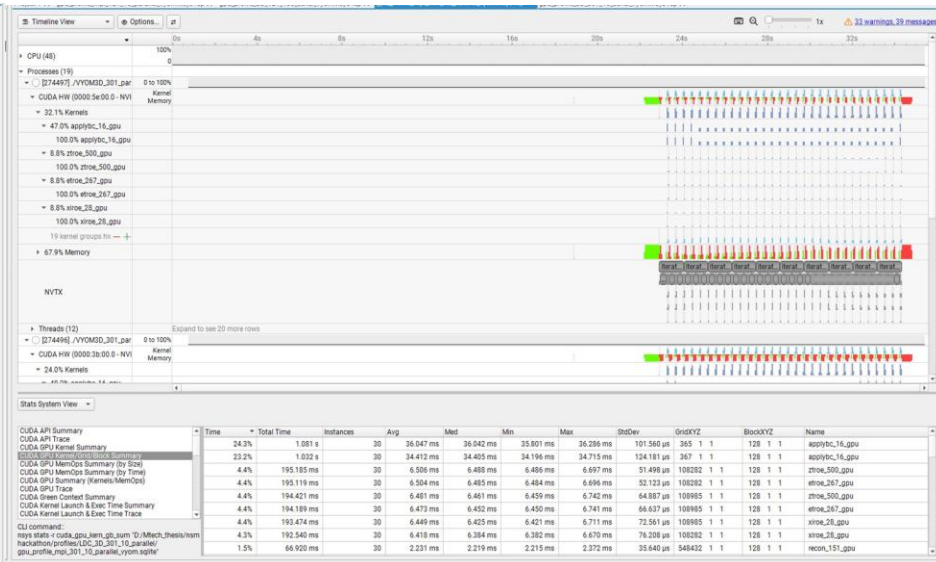
Feature	Original
Halo Packing	CPU
MPI	Blocking
Communication Overlap	No
GPU Utilization	Low
PCIe transfer	High

Nsight profiling Results

Parallel

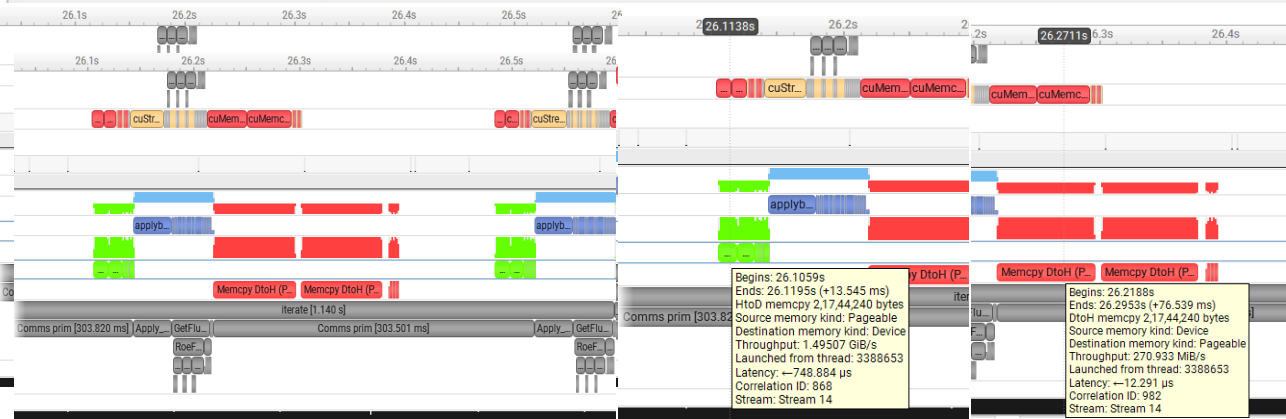
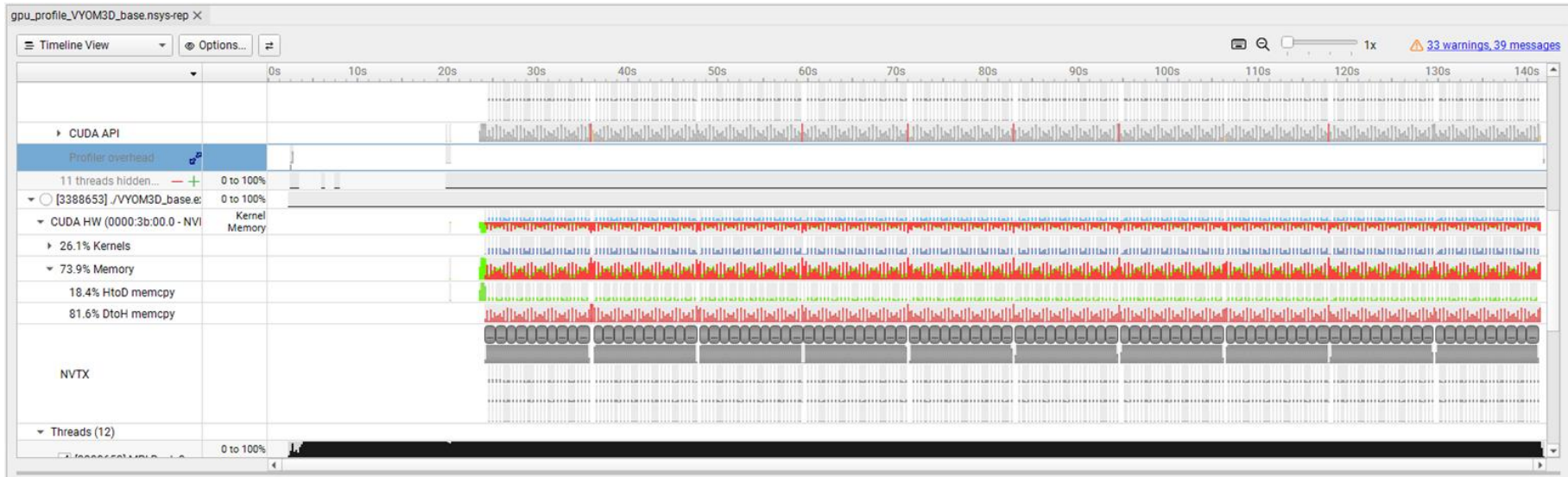


1.7 Million Grid (121 X 121 X 121)

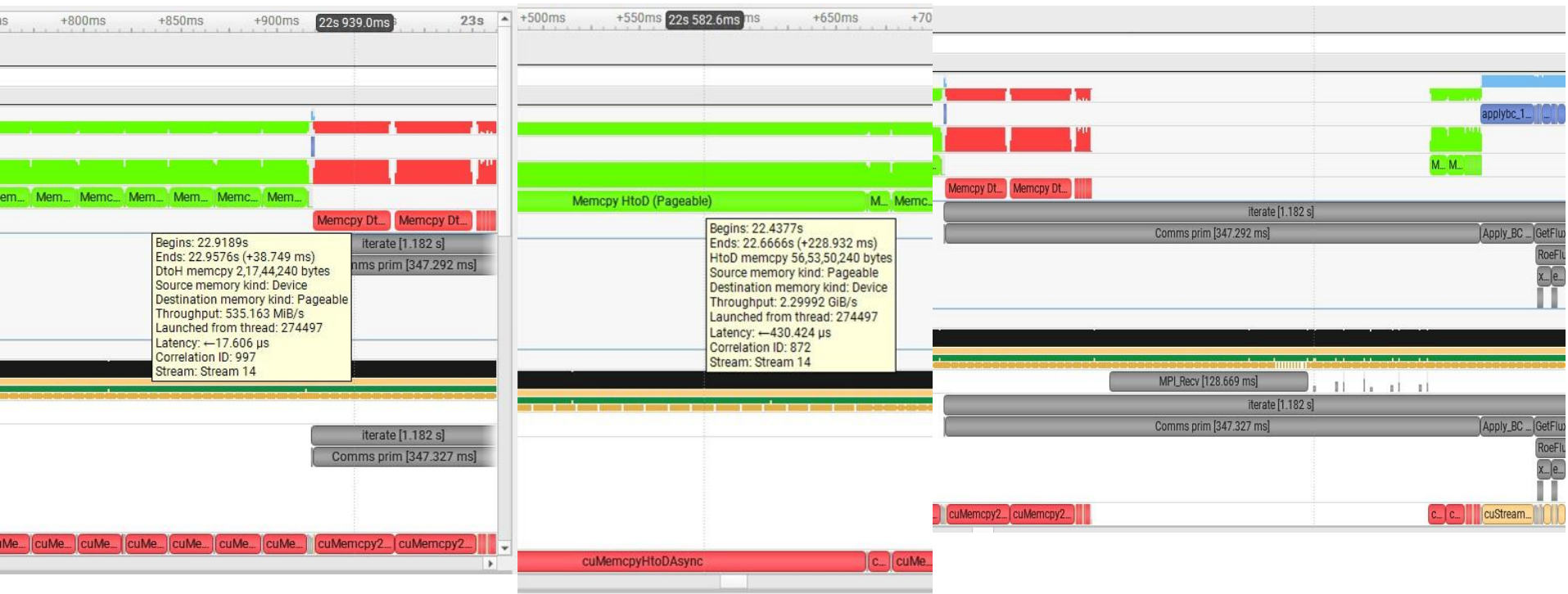


27 Million Grid (301 X 301 X 301)

Baseline profiling Results



Challenges - D2H and H2D Transfer



- Excessive Host to device transfer (Comms prim) - 347.327 ms
- Larger Memory timeline
- IDLE GPU
- Communication profile

CUDA GPU kernel Summary

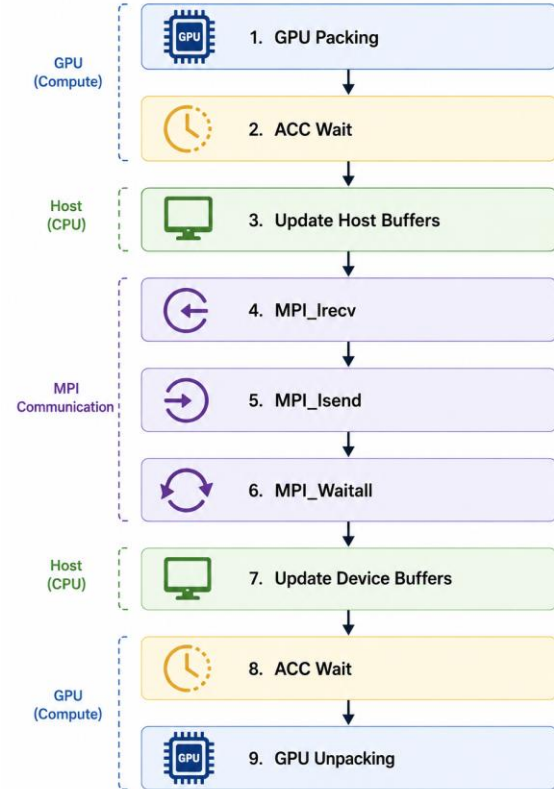
Stats System View											
CUDA API Trace	Time	Total Time	Instances	Avg	Med	Min	Max	StdDev	Name		
CUDA GPU Kernel Summary	47.6%	2.114 s	60	35.229 ms	35.258 ms	34.196 ms	36.286 ms	831.886 µs	applybc_16_gpu		
CUDA GPU Kernel/Grid/Block Summary											
CUDA GPU MemOps Summary (by Size)	8.8%	389.606 ms	60	6.493 ms	6.487 ms	6.459 ms	6.742 ms	59.479 µs	ztroe_500_gpu		
CUDA GPU MemOps Summary (by Time)	8.8%	389.308 ms	60	6.488 ms	6.484 ms	6.450 ms	6.741 ms	61.341 µs	etroe_267_gpu		
CUDA GPU Summary (Kernels/MemOps)											
CUDA GPU Trace	8.7%	386.014 ms	60	6.434 ms	6.423 ms	6.382 ms	6.711 ms	75.422 µs	xiroe_28_gpu		
CUDA Green Context Summary	3.0%	133.610 ms	60	2.227 ms	2.216 ms	2.208 ms	2.372 ms	35.349 µs	recon_151_gpu		
CUDA Kernel Launch & Exec Time Summary											
CUDA Kernel Launch & Exec Time Trace	3.0%	132.908 ms	60	2.215 ms	2.205 ms	2.197 ms	2.354 ms	33.878 µs	recon_92_gpu		
CUDA Summary (API/Kernels/MemOps)											
DX11 PIX Range Summary	2.9%	129.462 ms	60	2.158 ms	2.157 ms	2.144 ms	2.173 ms	6.935 µs	roeflux_59_gpu		
DX12 GPU Command List PIX Ranges Summary	2.8%	126.423 ms	60	2.107 ms	2.105 ms	2.068 ms	2.216 ms	38.392 µs	recon_34_gpu		
DX12 PIX Range Summary											
MPI Event Summary	1.8%	80.751 ms	60	1.346 ms	1.346 ms	1.338 ms	1.354 ms	4.246 µs	iterate_44_gpu		
MPI Event Trace	1.5%	65.185 ms	60	1.086 ms	1.086 ms	1.079 ms	1.094 ms	4.428 µs	secondvflux_ztvsecond_198_gpu		
MPI Message Size Summary											
NVTX GPU Projection Summary	1.5%	64.897 ms	60	1.082 ms	1.082 ms	1.074 ms	1.090 ms	4.579 µs	secondvflux_etvsecond_144_gpu		
NVTX GPU Projection Trace	1.4%	64.090 ms	60	1.068 ms	1.068 ms	1.061 ms	1.074 ms	3.850 µs	secondvflux_xivsecond_90_gpu		
NVTX Push/Pop Range Summary											
NVTX Push/Pop Range Trace	1.3%	57.715 ms	60	961.910 µs	962.059 µs	949.129 µs	974.155 µs	5.314 µs	secondvflux_etvsecond_112_gpu		
NVTX <u>nvtx_pushpop_summary</u>											
NVTX Range Summary	1.3%	57.082 ms	60	951.360 µs	949.963 µs	941.577 µs	965.674 µs	5.985 µs	secondvflux_ztvsecond_166_gpu		
CLI command:	1.3%	55.768 ms	60	929.466 µs	929.786 µs	926.217 µs	933.066 µs	3.137 µs	roeflux_13_gpu		
nsys stats -r cuda_gpu_kern_sum 'C:\\Users\\mital\\AppData\\Local\\Temp\\tmp-27900-co0uHVPb26FN\\'gpu_profile_mpi_301.sqlite'	1.2%	54.867 ms	60	914.443 µs	914.825 µs	909.224 µs	919.722 µs	3.379 µs	secondvflux_xivsecond_58_gpu		
	1.2%	51.195 ms	60	853.254 µs	853.289 µs	850.376 µs	856.298 µs	2.847 µs	roeflux_9_gpu		

NVTX Range Summary

[illegible]

Code Optimization & Implementation

Proposed GPU communication profile



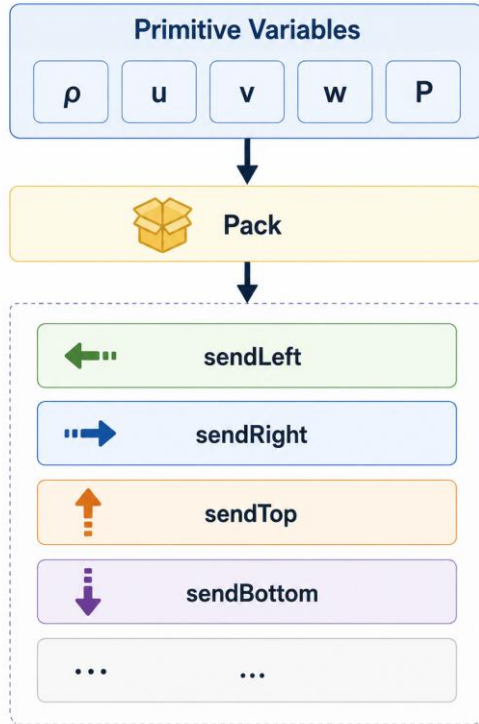
Features :

1. GPU packing
2. GPU unpacking
3. Non-blocking MPI
4. Separate communication buffers
5. OpenACC data regions

Major Code changes

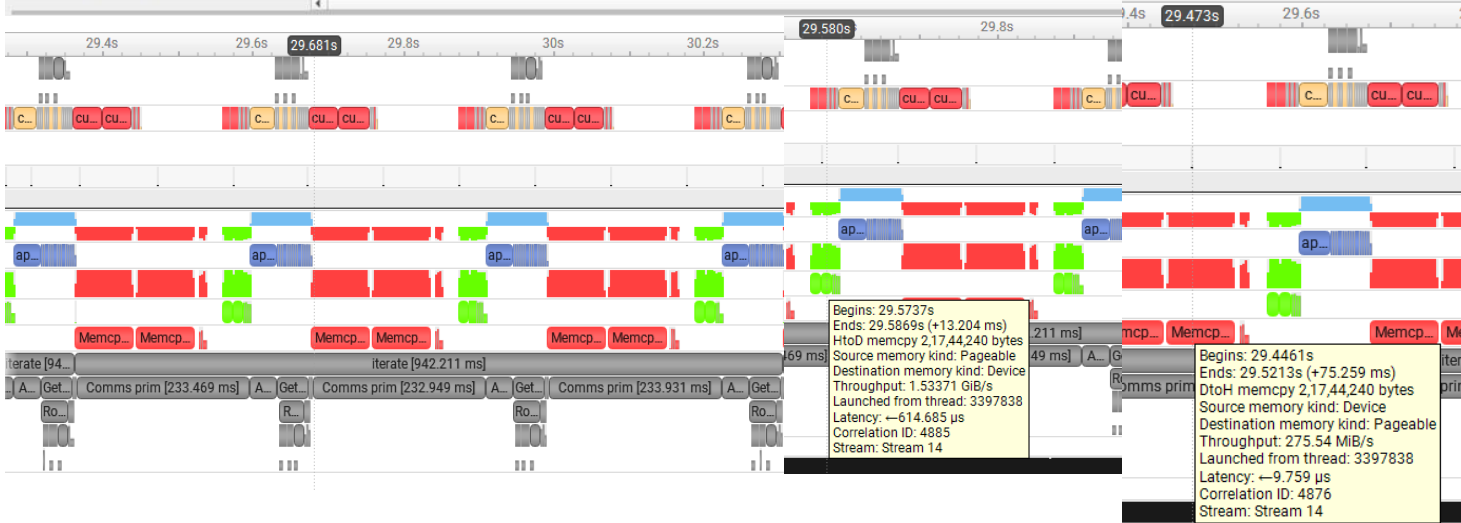
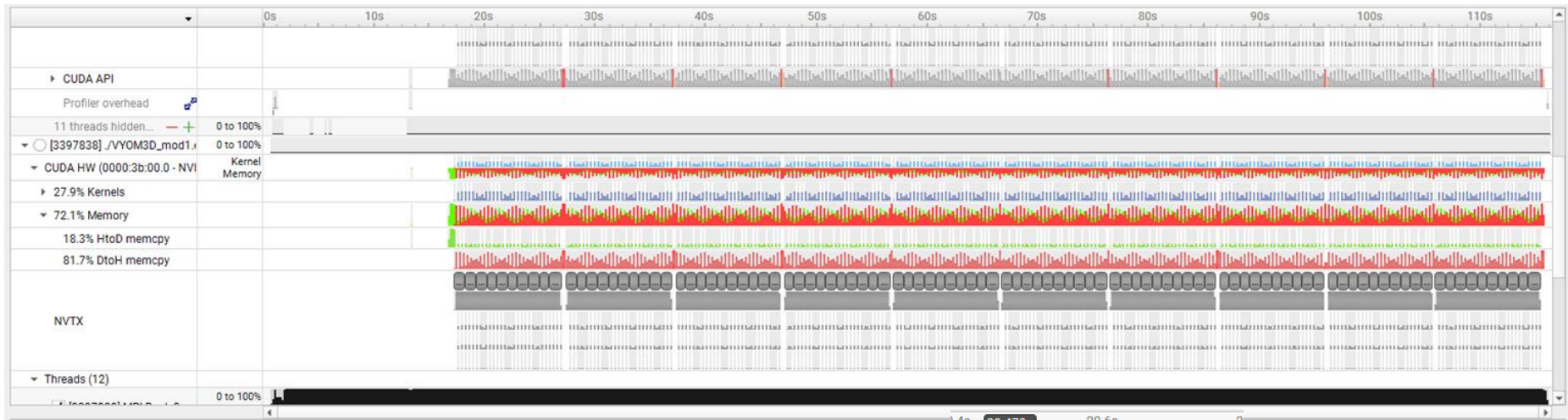
File	Modification
iterate.f90	Replaced Communicate() with Communicate_Prim()
Communicate_Prim.f90	GPU packing/unpacking + MPI_Isend/Irecv
mod_spmd.f90	Communication buffers added
AllocateVar.f90	GPU communication buffers allocated
Main3D.f90	Updated initialization communication
Output3D.f90	Buffer compatibility updates

GPU Packing Strategy and Communication Optimization Summary



Optimization
GPU Packing
GPU Unpacking
Non-blocking MPI
Separate Communication Buffers
GPU Resident Buffers
CPU Packing Removed

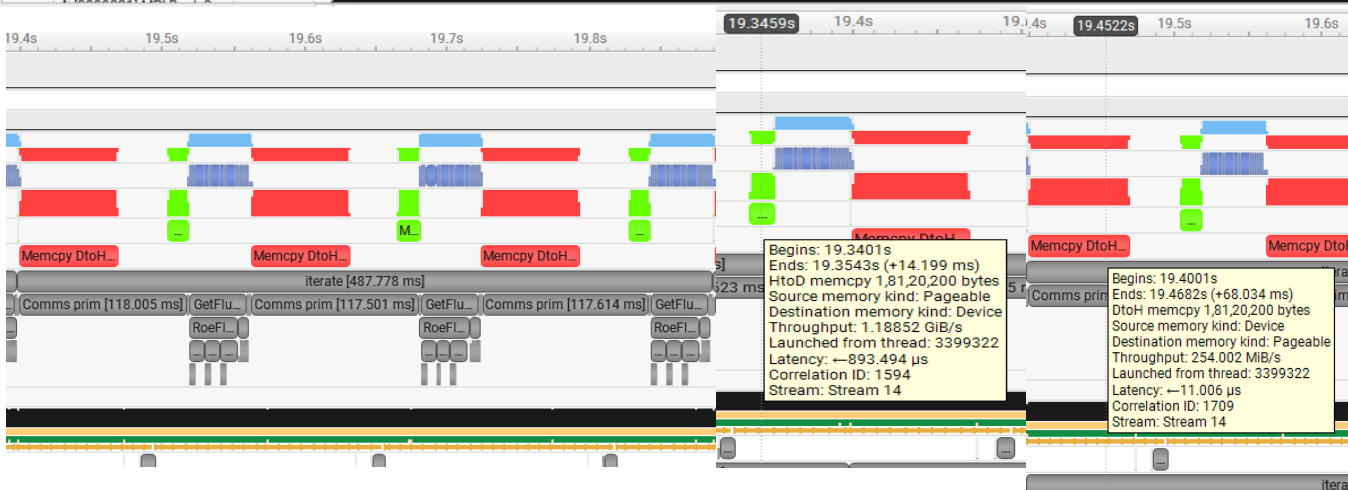
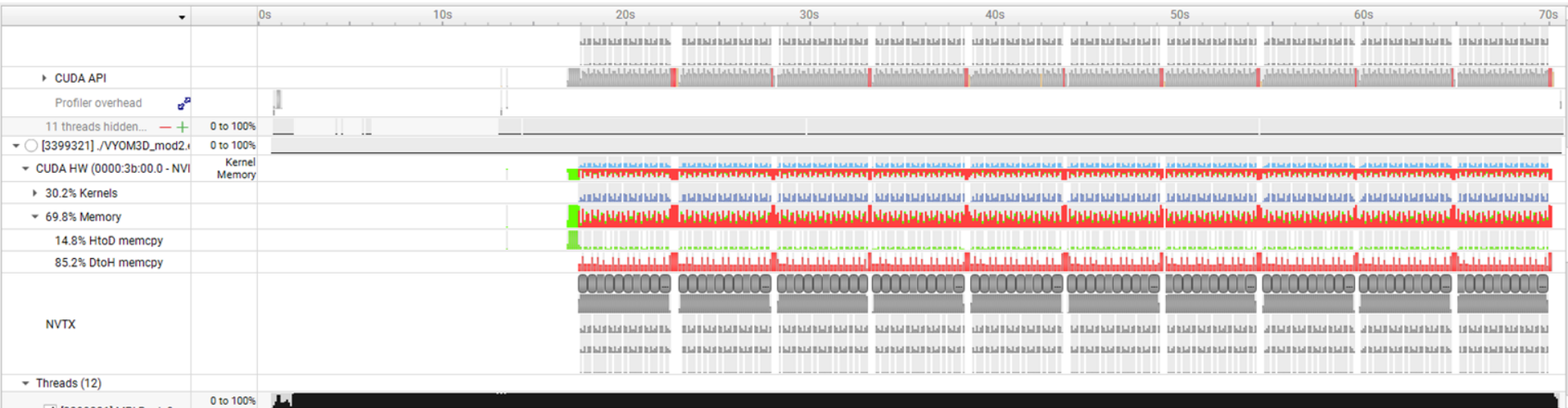
Mod 1



Time for 100 iteration:
97.6 s

Speed up : 1.18x

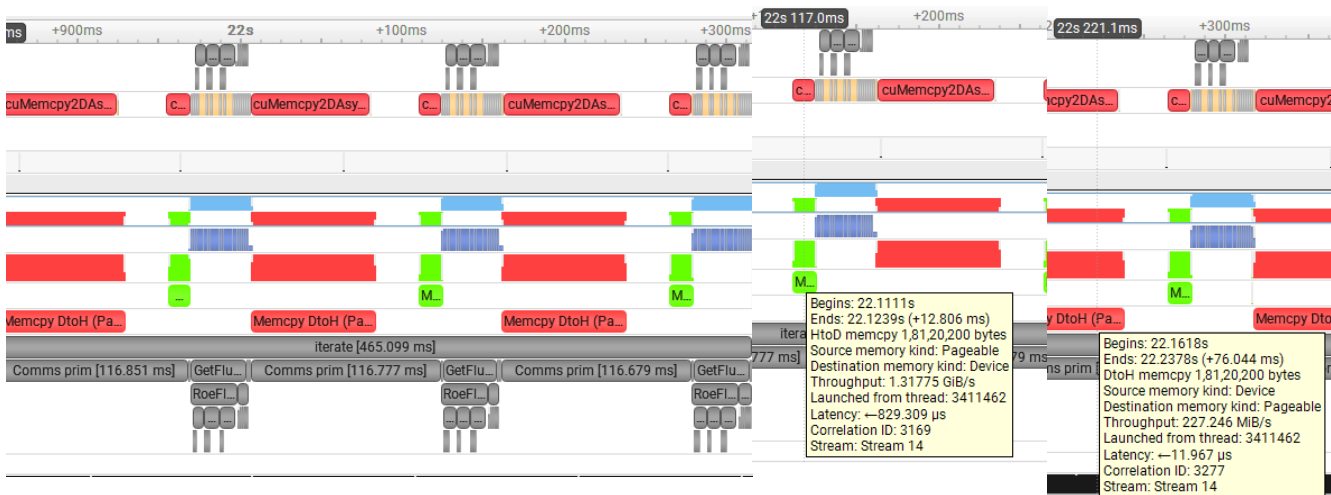
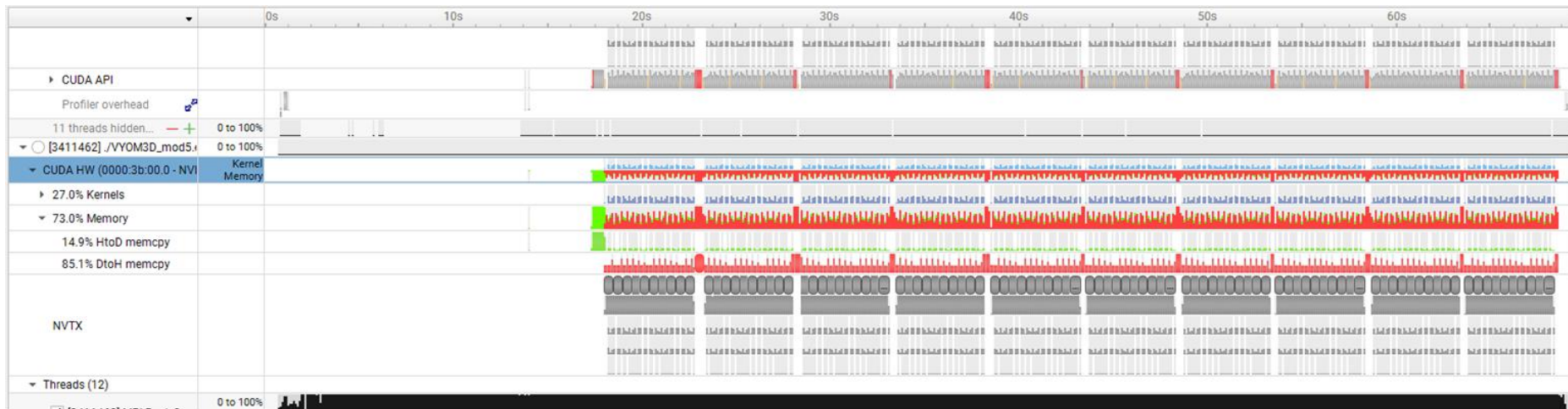
Mod 2



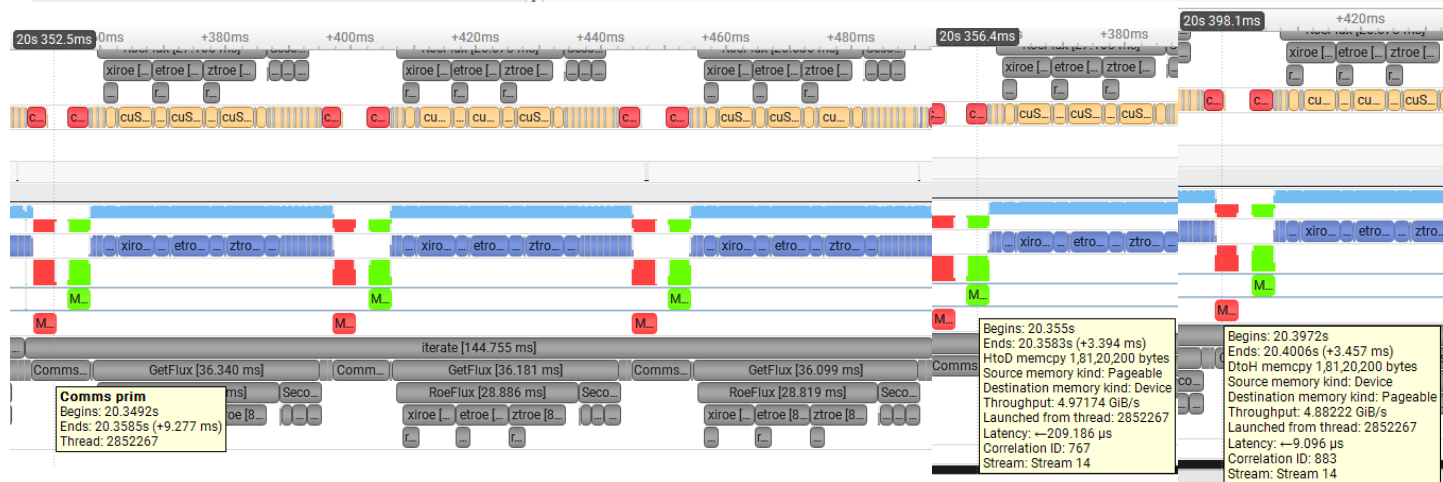
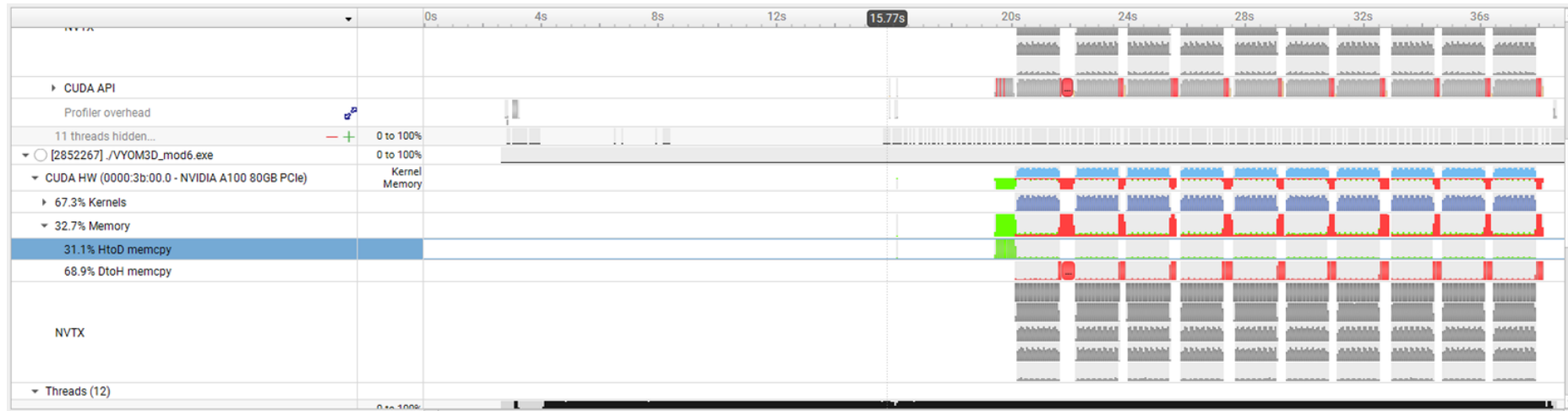
Time for 100 iteration: 52.1 s

Speed up : 2.2x

Mod 3



Mod 4



Time for 100 iteration:
17.55 s

Speed up : 6.6x

Experimental packing Data Sets

Version	Wall time (100 iterations, 2 GPUs)	Speedup vs baseline	Correctness/ Problems
Baseline (VYOM, unmodified)	115.2 s	1.0x	Reference
Optimized 4D Packed Buffer Communication	42.14 s	2.73x	Out-of-bounds ghost-cell writes and boundary-condition indexing issues were resolved, restoring numerically stable and physically consistent solutions.
3D Temporary Buffer (Arrays) Slicing	40.91 s	2.81x	Reduced communication time but communication still involved larger temporary buffers, resulting in higher memory usage.
2D Temporary Buffer (Arrays) Slicing	25.52 s	4.51x	Better communication performance. Numerical correctness need to be validated due to packing/unpacking and halo synchronization

Performance Data Sets

Version	Wall time (100 iterations, 2 GPUs)	Speedup vs baseline	Correctness/ Problems
Baseline (VYOM, unmodified)	115.2 s	1.0x	Reference
Mod 1- Intermediate optimization (packed comms, first pass)	97.6 s	1.18 x	Diverged (NaN by iteration) but still faster
Intermediate optimization (packed comms, first pass)	52.1 s	2.2x	Diverged (NaN by iteration) but still faster
Corrected Communication (exact-range halo update, explicit sync)	50.0 s	2.3x	Solves and matches residuals with baseline
Corrected Communication & boundary conditions fix	17.55s	6.6x	Solves and matches residuals with baseline

Base → Mod1 : Packing Halo Communication

File: Communicate.f90 → new file CommunicateAll.f90

BEFORE

```
subroutine Communicate(arrayin, arrayout)
  real(8), intent(in)  :: arrayin(nx,ny,nz)
  real(8), intent(out) :: arrayout(nx,ny,nz)

  arrayout = arrayin      ! full 218 MB copy
                          ! every call, every variable
  call MPI_send(arrayout(...), ..., &
                pright, 15, mpicom, ierr)
  call MPI_recv(arrayout(...), ..., &
                pleft, 15, mpicom, status, ierr)
  ! repeated: BLOCKING, once per variable
  ! 5 vars x 3 RK stages = 15 calls/iteration
```

AFTER

```
! CommunicateAll: ALL 5 variables in ONE buffer

call MPI_ISEND(sbuf_xl, nxsz, mpir8, pleft, &
              TAG_XMINUS, mpicom, reqs(nreq), ierr)
call MPI_Irecv(rbuf_xl, nxsz, mpir8, pleft, &
              TAG_XPLUS, mpicom, reqs(nreq), ierr)
! ... non-blocking sends/recvs, all faces ...

call MPI_WAITALL(nreq, reqs, statuses, ierr)
```

1. 5 messages → 1 packed message per face direction
2. Blocking Send/Recv → non-blocking Isend/Irecv + single Waitall

Mod1 → Mod2 : Faster, But Broken

Same packed-communication approach, tuned further — no new bug, the old one just wasn't caught yet

Run Log — identical failure signature in BOTH mod1 and mod2

itr	min (u)	min (p)	norm (rhs)
0	0.0000000	17.8571429	-16.0000000
10	—	Inf	NaN

Root cause carried forward from mod1, unchanged:

1. Host copy (!\$acc update self) used APPROXIMATE index ranges
2. No guarantee the GPU had finished WRITING that data first
3. base's slowness accidentally gave the GPU time to finish — masking the race
4. faster comms removed that accidents — race became visible as NaN

Mod2 → Mod3 : Finding & Fixing The Race

File: CommunicateAll.f90 (rewritten) — exact ranges + explicit sync + phased exchange

BEFORE

```
! Approximate range, no explicit wait,  
! X/Y/Z all packed together in one flat pass  
!$acc update self(arr(1:nx, &  
    jegrid-noverlap+1:jegrid, 1:nz, 1:nv))  
  
! ... pack X,Y,Z together, fire all sends/recvs,  
!     ONE big waitall at the very end
```

AFTER

```
!$acc wait      ! guarantee GPU kernel finished  
!$acc update self(arr(1:nx, &  
    jegrid-noverlap+1:jegrid, 1:nz, 1:nv))  
    ! exact isgrid/iegrid-based range  
  
! PHASE 1: X-direction (fully completes)  
! PHASE 2: Y-direction (sees fresh X corners)  
! PHASE 3: Z-direction (sees fresh X+Y corners)
```

1. Exact isgrid/iegrid ranges instead of generic approximate ranges
2. Explicit !\$acc wait before every host read — no more relying on lucky timing
3. Split into 3 SEQUENTIAL phases (X → Y → Z) so corner data propagates correctly

Mod3 → Mod4 : The Big Performance Win

File: ApplyBC.f90 — bug present in base, mod1, mod2, AND mod4

BEFORE

```
do k = 1, nz
  do j = 1, 1
    do i = 1, nx
      prim(:,1,:,1) = prim(:,2,:,1) ! ':' = WHOLE
      prim(:,1,:,2) = 0.0d0         ! ARRAY, inside
      prim(:,1,:,3) = 0.0d0         ! an i,k loop!
      prim(:,1,:,4) = 0.0d0         ! Runs ~90,600x
      prim(:,1,:,5) = prim(:,2,:,5) ! instead of once
    end do
  end do
end do
```

AFTER

```
do k = 1, nz
  do j = 1, 1
    do i = 1, nx
      prim(i,1,k,1) = prim(i,2,k,1) ! indexed by
      prim(i,1,k,2) = 0.0d0         ! the actual
      prim(i,1,k,3) = 0.0d0         ! loop
      prim(i,1,k,4) = 0.0d0         ! variables
      prim(i,1,k,5) = prim(i,2,k,5)
    end do
  end do
end do
```

1. The ':' means “every i, every k” — but this line already sits inside a loop over i and k which reduced 48.9% of GPU in base case(35 ms/call) to 0.5% of GPU kernel. (180 μ s/call)
2. Every one of the 301×301 loop passes redundantly re-assigns the ENTIRE face again

Benchmark Datas

units	ms	ms	GiB/s	MiB/s	s
	Iterate	CommsPrims	Memory HtoD	Memory DtoH	Total Time to do 100 iterations with 2 GPUs
Baseline	1181	342.638	1.49	270.93	115.2
Mod1	942.11	232.949	1.53	275.54	97.6
Mod2	487.78	118	1.18	254	52.1
Mod3	465.509	116.851	1.317	227.246	25.52
Mod4	144.755	9.277	4.97	4882	17.55

Risks and Challenges

Memory and Data management

1. Bandwidth limits and transfer overhead in high-order reconstruction
2. Temporary array allocation overhead across subroutines

Multi-GPU Scaling

1. Halo exchange & inter-GPU communication overhead
2. Load imbalance and latency/sync costs limits scaling to more GPUs

Next Steps

1. Further Optimize the communication framework
2. Validate larger mesh sizes without numerical divergence

References

1. Ranjan, R., Unnikrishnan, S. and Gaitonde, D., 2020. A robust approach for stability analysis of complex flows using high-order Navier-Stokes solvers. *Journal of Computational Physics*, 403, p.109076.
2. Giannetti, F., Luchini, P. and Marino, L., 2009, September. Linear stability analysis of three-dimensional lid-driven cavity flow. In Atti del XIX Congresso AIMETA di Meccanica Teorica e Applicata (pp. 14-17). Aras Edizioni Ancona.
3. Visbal, M.R. and Gaitonde, D.V., 2002. On the use of higher-order finite-difference schemes on curvilinear and deforming meshes. *Journal of Computational Physics*, 181(1), pp.155-185.
4. Castro, M., Costa, B. and Don, W.S. (2011). High order weighted essentially non-oscillatory WENO-Z schemes for hyperbolic conservation laws. *Journal of Computational Physics*, [online] 230(5), pp.1766–1792. doi:10.1016/j.jcp.2010.11.028.
5. Borges, R., Carmona, M., Costa, B. and Don, W.S. (2008). An improved weighted essentially non-oscillatory scheme for hyperbolic conservation laws. *Journal of Computational Physics*, [online] 227(6), pp.3191–3211. doi:10.1016/j.jcp.2007.11.038

Next Step and Timeline

Next Steps

1. Temporary array allocation and data transfer to reduce overall d2h and h2d time
2. Proper packing and unpacking of data through MPI Communication
3. GPU-side halo packing with persistent communication buffers to eliminate repeated host-side packing and buffer allocations.
4. Validation of Lid Driven Cavity case for the developed Multi - GPU code.

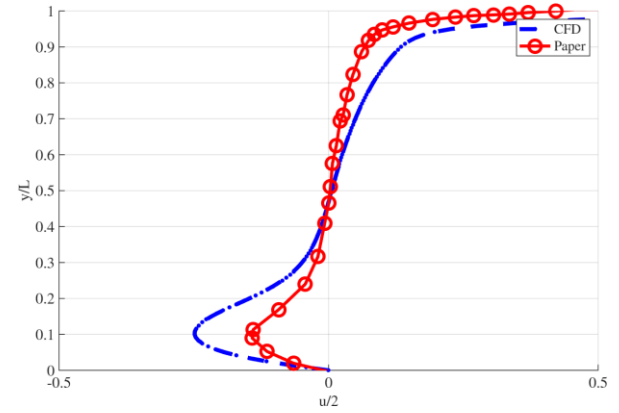
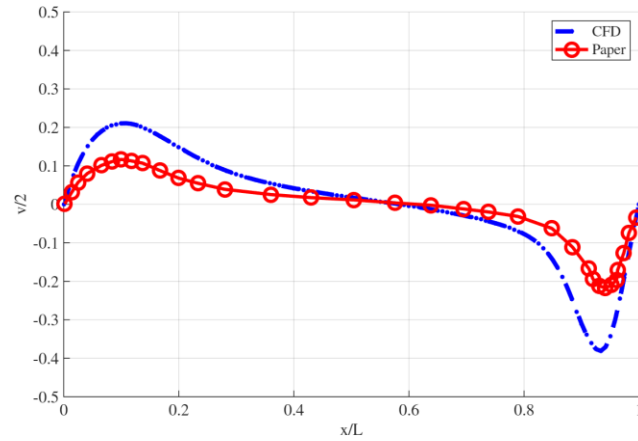
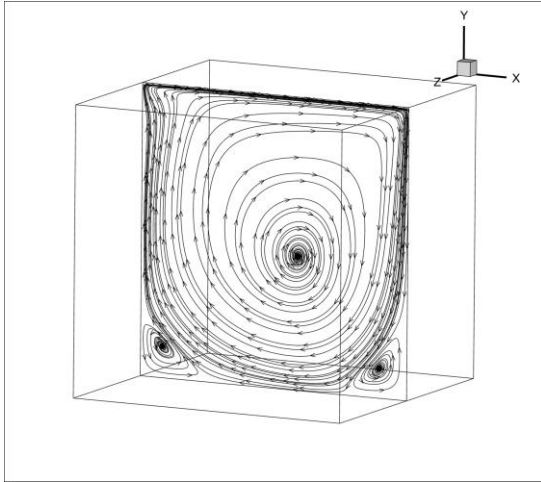
Acknowledgement

- **The NSM Month-Long Hackathon Organizing Committee** for providing the opportunity, computational resources, and GPU infrastructure to carry out this work.
- **The mentors from IIT Bombay and C-DAC Pune** for their valuable technical guidance, insightful discussions, and continuous support throughout the hackathon.
- **Mr. Mandar Gurav (Team Mentor)** for his constant mentoring, technical expertise, and guidance in code optimization and performance profiling.
- **Dr. Rajesh Ranjan (Team Leader & Faculty Guide, IIT Kanpur)** for his invaluable guidance, encouragement, and continuous support throughout the project.

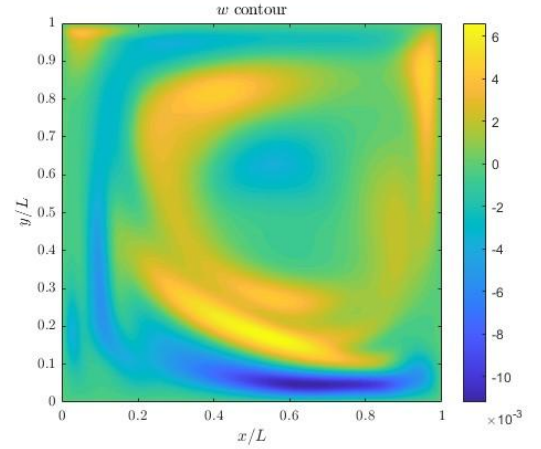
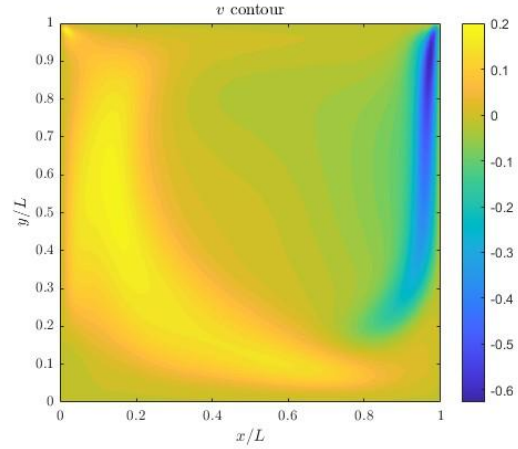
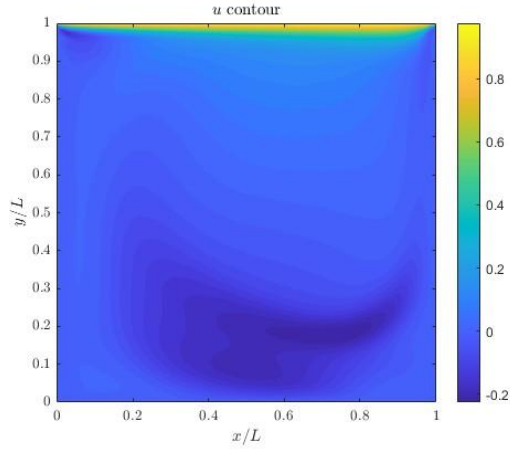
THANK YOU

APPENDIX

3D-Lid Driven Cavity



3D-Lid Driven Cavity



MULTI-GPU 3D PARALLEL RUN SIMULATION RESULTS

